

Condizioni in C++

Condizioni in C++

Prendere decisioni nel codice con if, else if, else e switch in C++.

Perché le condizioni?

Un programma che fa sempre la stessa cosa non è molto utile. I programmi interessanti prendono **decisioni**: se l'utente è maggiorenne, mostra il contenuto; se il voto è sufficiente, passa alla classe successiva; se piove, porta l'ombrello.

Le condizioni permettono al programma di scegliere tra strade diverse in base a una situazione.

L'istruzione **if**

if significa "se". Se la condizione tra parentesi è vera, il blocco di codice viene eseguito. Altrimenti viene saltato.

```
if (condizione) {  
    // questo codice viene eseguito solo se la condizione è vera  
}
```

```
int eta = 18;  
  
if (eta >= 18) {  
    cout << "Sei maggiorenne." << endl;  
}  
// Se eta fosse 15, questa riga non verrebbe mai stampata
```

Nota: in C++ la condizione **deve** stare tra parentesi tonde **()**.

L'istruzione `else`

`else` significa "altrimenti". Definisce cosa fare quando la condizione è falsa:

```
int eta = 15;

if (eta >= 18) {
    cout << "Sei maggiorenne." << endl;
} else {
    cout << "Sei minorenni." << endl;
}
// Stampa: "Sei minorenni."
```

L'istruzione `else if`

Quando hai più di due possibilità, usa `else if` per controllare più condizioni in sequenza:

```
int voto = 75;

if (voto >= 90) {
    cout << "Ottimo!" << endl;
} else if (voto >= 70) {
    cout << "Buono!" << endl;           // ← questa viene stampata
} else if (voto >= 60) {
    cout << "Sufficiente." << endl;
} else {
    cout << "Insufficiente." << endl;
}
```

Il programma controlla le condizioni dall'alto verso il basso e si ferma alla prima che risulta vera. Le successive vengono ignorate.

Condizioni annidate

Puoi mettere un `if` dentro un altro `if`. Pensa a esso come a una serie di domande: prima la domanda principale, poi le sotto-domande.

```

int eta = 20;
bool haPatente = true;

if (eta >= 18) {
    // siamo maggiorenni – ora controlliamo la patente
    if (haPatente) {
        cout << "Puoi guidare." << endl;
    } else {
        cout << "Sei maggiorenne ma non hai la patente." << endl;
    }
} else {
    cout << "Sei minorenne, non puoi guidare." << endl;
}

```

Usare gli operatori logici

Puoi combinare più condizioni in una sola usando `&&` (E), `||` (O) e `!` (NON):

```

int eta = 16;
bool conAccompagnatore = true;

// Può entrare se è maggiorenne OPPURE se ha un accompagnatore
if (eta >= 18 || conAccompagnatore) {
    cout << "Puoi entrare." << endl;
}

```

```

int temperatura = 22;
bool piove = false;

// Perfetto per uscire se è caldo E non piove
if (temperatura > 20 && !piove) {
    cout << "Perfetto per uscire!" << endl;
}

```

Il costrutto `switch`

Quando devi confrontare una variabile con molti valori fissi, `switch` è più leggibile di una lunga serie di `else if`. Funziona come un selettore: vai direttamente al caso corrispondente.

```
int giorno = 3;

switch (giorno) {
    case 1:
        cout << "Lunedì" << endl;
        break;
    case 2:
        cout << "Martedì" << endl;
        break;
    case 3:
        cout << "Mercoledì" << endl; // ← questo viene eseguito
        break;
    case 4:
        cout << "Giovedì" << endl;
        break;
    case 5:
        cout << "Venerdì" << endl;
        break;
    default:
        cout << "Weekend" << endl; // eseguito se nessun case
        corrisponde
}
```

Output: Mercoledì

I componenti di `switch`

- **case valore:** — esegue il codice se la variabile vale quel numero
- **break** — esce dallo switch. Senza di esso, l'esecuzione continua nei case successivi!
- **default** — viene eseguito se nessun case corrisponde (facoltativo)

Attenzione al `break` !

Dimenticare il `break` è un errore comune. Senza di esso, dopo aver trovato il case giusto il programma continua ad eseguire anche i case successivi:

```

int x = 2;
switch (x) {
    case 1:
        cout << "uno" << endl;
    case 2:
        cout << "due" << endl;    // eseguito (corretto)
    case 3:
        cout << "tre" << endl;    // eseguito anche questo! (manca break)
        break;
}
// Output: due
//         tre

```

A volte questo comportamento è intenzionale. Per esempio, se vuoi che più case eseguano lo stesso codice:

```

char voto = 'B';
switch (voto) {
    case 'A':
    case 'B':
        // sia A che B arrivano qui
        cout << "Promosso con lode" << endl;
        break;
    case 'C':
        cout << "Promosso" << endl;
        break;
    default:
        cout << "Bocciato" << endl;
}

```

L'operatore ternario

Per condizioni semplici con solo due risultati, esiste una forma compatta su una riga sola:

```

// condizione ? valore_se_vero : valore_se_falso
int eta = 20;
string stato = (eta >= 18) ? "maggiorenne" : "minorenne";
cout << stato << endl;    // maggiorenne

```

Usa questa forma solo quando è chiara da leggere. Se la condizione è complessa, preferisci un normale `if / else` .

Esempio pratico

```
#include <iostream>
using namespace std;

int main() {
    int punteggio;
    cout << "Inserisci il punteggio (0-100): ";
    cin >> punteggio;

    // Prima controlliamo che il valore sia valido
    if (punteggio < 0 || punteggio > 100) {
        cout << "Punteggio non valido!" << endl;
    } else if (punteggio >= 90) {
        cout << "Voto: A – Eccellente!" << endl;
    } else if (punteggio >= 80) {
        cout << "Voto: B – Molto buono" << endl;
    } else if (punteggio >= 70) {
        cout << "Voto: C – Buono" << endl;
    } else if (punteggio >= 60) {
        cout << "Voto: D – Sufficiente" << endl;
    } else {
        cout << "Voto: F – Insufficiente" << endl;
    }

    return 0;
}
```